```
# Apply inverse QFT
qc_test.compose(inverse_qft(3), inplace=True)

sv = Statevector.from_instruction(qc_test)
print(sv)
Statevector([-9.52420783e-18-8.34944624e-18j,
             7.07106781e-01+8.34944624e-18j,
            -9.52420783e-18-1.32994552e-17j,
             7.07106781e-01+1.32994552e-17j,
             3.36731597e-18+2.34892061e-34j,
             3.36731597e-18-3.09843189e-34j,
             3.36731597e-18+4.76210391e-18j,
             3.36731597e-18-4.76210391e-18j],
           dims=(2, 2, 2))
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt

qc_meas = QuantumCircuit(3,3)

qc_meas.x(1)

qc_meas.compose(qft(3), inplace=True)

qc_meas.measure(range(3), range(3))

sim = AerSimulator()
result = sim.run(qc_meas, shots=1024).result()
counts = result.get_counts()

print(counts)
plot_histogram(counts)
#plt.show()
{'101': 109, '100': 128, '110': 138, '001': 129, '111': 135, '011':
136, '000': 118, '010': 131}
No description has been provided for this image
```

1. First — What is Quantum Fourier Transform (QFT)?

Think of **QFT as a "pattern detector"**.

- In classical computing, Fourier Transform converts a signal into frequencies.

- In quantum computing, **QFT converts a quantum state into phase (frequency-like) information**.

👉 Simple analogy:

- Input = "numbers in time"
- Output = "hidden frequency pattern"

---

2. Core Theory (very important for viva)

QFT transforms a quantum state like this:

$$|x\rangle \rightarrow \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i x k/N} |k\rangle$$

Meaning in simple words:

- You start with a state **|x⟩**
- QFT spreads it into **all possible states**
- Each state gets a **phase (rotation)**

👉 That phase is where the "information" is stored.

---

3. Now Your Code — Explained Simply

🔹 PART 1: Manual QFT (2 Qubits)

```python
qc = QuantumCircuit 2
```

👉 Create 2 qubits

Step 1: Hadamard

```python
qc.  0
```

👉 Makes qubit 0 into superposition:

- |0⟩ → (|0⟩ + |1⟩)

Step 2: Controlled Phase

```python
qc.   π/2  1  0
```

👉 This is the MOST IMPORTANT step.

- If qubit 1 = 1 → rotate qubit 0 by **π/2**
- If qubit 1 = 0 → do nothing

👉 This adds **phase information**

Step 3: Hadamard on qubit 1

```python
qc.  1
```

👉 Same idea:

- Creates superposition on second qubit

Step 4: Swap

```python
qc.     0 1
```

👉 QFT reverses qubit order

- Output becomes reversed → so we fix it

🔹 Input Testing

```python
qc_test.  1   # |10>
```

👉 Input state:

- qubit 1 = 1
- qubit 0 = 0

So state = **|10⟩**

🔹 Output Statevector

```
[redacted]
```

👉 Interpretation:

- All states appear
- But each has **different phase (sign / imaginary part)**

👉 This is exactly what QFT does:
✔️ spreads amplitudes
✔️ encodes information in phase

🔹 PART 2: General QFT (n qubits)

```python
for j in range n
    qc.  j
```

👉 Apply Hadamard to each qubit

```python
qc.   π / 2^ k-j   k  j
```

👉 Controlled rotations with decreasing angles:

Distance	Rotation
1 step	π/2
2 step	π/4
3 step	π/8

👉 Key idea:

- Far qubits → smaller influence

Swap Loop

```python
```

```
for i in range n//2
    qc.     i  n-i-1
```

👉 Reverse qubits (important detail for correctness)

🔹 PART 3: Inverse QFT

👉 This undoes QFT.

Key differences:

- Reverse order
- Negative angles

```python
qc.   -θ
qc.  j
```

👉 So:

QFT	Inverse QFT
+θ	−θ
forward	reverse

🔹 Verification

```python
QFT → inverse QFT
```

👉 Output ≈ original state

✔ This proves correctness
✔ Small errors = floating point noise

🔹 PART 4: Measurement Output

```
```

👉 Almost uniform distribution

Why?

- QFT spreads amplitude across all states
- Measurement collapses randomly

👉 So you see:
✔️ all states appear
✔️ nearly equal probability

🔥 Important Concepts for Viva

---

1. Why QFT is useful?

QFT is used in:

- Shor's Algorithm (factorizing numbers)
- Quantum Phase Estimation

👉 It extracts **periodicity / phase**

---

2. Key Gates Used

Hadamard Gate

- Creates superposition

Controlled Phase Gate

- Adds phase shift

Swap Gate

- Fixes qubit order

---

3. Time Complexity

- Classical Fourier Transform: **O(N log N)**
- QFT: **O(n^2)** gates

👉 Much faster for quantum states

---

4. Key Insight (VERY IMPORTANT)

👉 QFT does NOT change probabilities directly
👉 It changes **phases**

And:

👉 Phase = hidden information in quantum computing

---

5. Common Mistake (You should avoid)

You might think:

> "Why output looks random?"

❌ Wrong assumption
✔️ It is NOT random
✔️ It encodes structured phase information

---

6. Debugging Understanding (important for you)

If QFT is confusing, check:

1. Input state ($|x\rangle$)
2. After Hadamard → superposition?
3. After CP → phase added?
4. After swap → correct order?

---

7. One-Line Intuition

👉 QFT = "convert quantum state into phase-frequency representation"

If you still feel unclear

Tell me:

- Which part? (Hadamard / CP / formula / output)

I can:

- ↳ break it into diagrams
- or ↳ simulate step-by-step amplitudes
- or ↳ give viva-ready answers only